The Machine, Part 2: Machine se Baat

August 25, 2026

CONTENTS

THE MACHINE

PART 2 OF 5: MACHINE SE BAAT

```
PAANCH PARTS KA NAKSHA

PART 1  PAISA AUR MACHINE       ho gaya
[ AAP YAHAN HO ]
PART 2  MACHINE SE BAAT         languages, operating system,
                                software banta kaise hai
PART 3  MACHINES KA JAAL        network, address, internet
PART 4  DATA AUR YAADEIN        storage, database, dhoondhna
PART 5  DUNIYA KO SERVE KARNA   scale, cloud, kharcha, poora jod
```

Pehle Part 1 ka imtihaan

Paanch sawal, Part 1 se. Pehle khud jawab do, phir neeche milaao. Jo atke, uska chapter number saath likha hai, wahin laut jao.

1. Paisa sirf Level 1 pe kyun ghusta hai? (0.3)
2. Formula mein ghante kyun nahi hain? (0.2)
3. Computer “sochta hai” kehna galat kyun hai? (1.3, 1.5)
4. Ek hi machine camera, bank aur game kaise ban jaati hai? (1.6)
5. Text se video lakh guna bhaari kyun hai? (2.1, 2.2)

> **Jawab:** 1. Kyunki zaroorat sirf insaan ki hoti hai; Level 2, 3, 4 aapas mein jo paisa dete hain, woh sab neeche se aaya hota hai. 2. Kamaai = size x scarcity x leverage; ghante bechna leverage ka sabse neecha dial hai. 3. Machine pehle se design ki hui gates ki chain chalati hai; har input ka output pehle se tay hai, surprise sirf hamare liye hai. 4. Kyunki recipe bhi number hai; nayi recipe daalo, machine wahi, kaam naya. 5. Akshar 1 byte hai; photo lakhon pixels x 3 numbers; video 30 photos prati second. Ginti khud guna hoti jaati hai.

Is part mein kya hai

Part 1 ne dikhaya ki machine numbers pe recipe chalati hai. Ab do sawal kholne hain:

Section 3: Recipe likhi kaise jaati hai? Insaan ke shabdon se machine ke numbers tak tarjuma kaun karta hai? Languages itni saari kyun hain? Aur woh manager kaun hai jo hazaar recipes ko ek machine pe ladne se rokta hai?

Section 4: Software BANTA kaise hai? Bug hote hi kyun hain? Badlav ka record kaise rehta hai? Aur engineer ka estimate hamesha galat kyun hota hai?

Aakhri wala chapter aapke liye sabse mehnga hai: jab aap kisi se software banwaoge, yeh part aapko batayega ki kya normal hai, kya laaparwahi, aur paisa kis cheez ka dena chahiye.

SECTION 3: MACHINE KO BATANA KI KYA KARNA HAI

Saat chapters. Insaan ki soch se machine ke numbers tak ka poora raasta: languages, translators, aur operating system.

Yeh aapke business decision mein kahan aayegi: “Kis language mein banayein,” “kaunsa platform pakdein,” “yeh muft tool bharosemand hai?” Yeh sab founder ke faisle hain, engineer ke nahi, kyunki inse hiring, kharcha aur raftaar tay hoti hai. Is section ke baad aap yeh faisle samajh ke le paoge, sun ke nahi.

Chapter 3.1 [SPINE]: Translation problem

Part 1 ka aakhri sach yeh tha: CPU sirf numbers wale kadam samajhta hai (machine code). 3 matlab jodo, 7 matlab chhalaang. Bas.

Toh 1940s ke pehle programmers ne kya kiya? Sach mein numbers hi likhe. Kaagaz pe kadam sochte the, phir har kadam ka number tay karte the, phir switchon aur cards se machine mein bharte the. Ek chhoti si recipe mein hafta lagta tha, aur ek number galat toh sab kachra, aur galti DHOONDHNA likhne se bhi mushkil.

Ab aap 1950 ke engineer ho. Yeh dard roz jhel rahe ho. Kya karoge?

Pehla kadam khud dikh jaata hai: numbers ke NAAM rakh do. “3” ki jagah likho “ADD”, “7” ki jagah “JUMP”. Insaan naam padh sakta hai, galti dikh jaati hai. Lekin machine toh naam nahi samajhti… toh ek chhota program likho jo naam ko wapas number bana de. Yeh tarjuma itna seedha hai (har naam = ek number) ki program aasaan tha.

Ruko. Yahan jo hua woh dhyaan se dekho, kyunki yeh poori software duniya ka janam hai: **tarjuma karne ka kaam khud ek recipe ban gaya.** Machine apni hi bhasha ka pul khud banati hai.

Ab agla dard. Naam number se behtar hain, lekin sochna ab bhi machine ke level pe padta hai: “yeh uthao, wahan rakho, jodo, chhalaang maaro.” Insaan aise nahi sochta. Insaan sochta hai: “har customer ka bill jodo aur sabse bada dikhao.” Ek line, jisme machine ke sau kadam chhupe hain.

Toh sapna bana: aisi bhasha jo insaan ki soch ke PAAS ho, aur ek mota translator jo us ek line ko machine ke sau kadmon mein khol de. Yeh translator ab seedha naam-badalna nahi hai, yeh asli tarjuma hai: samajhna, todna, jamana. Mushkil program hai. Lekin ek baar ban jaaye (leverage yaad hai?), toh har programmer hamesha ke liye insaan ke level pe likh sakta hai.

Yeh ban gaya (1957 se shuru), aur tab se poori industry isi seedhi pe upar chadh rahi hai: machine se door, insaan ke paas. Aakhri kadam aap roz dekhte ho: aap Claude ko HINDI mein bolte ho aur code ban jaata hai. Woh isi 70 saal purani seedhi ka sabse

naya danda hai.

NAAM

Numbers wali bhasha: **machine code**. Naam wali: **assembly**. Insaan ke paas wali bhaashaein: **programming languages** (Python, Java, C, aage aayengi). Tarjuma karne wala program: **translator** (iske do roop Chapter 3.3 mein). Is seedhi mein “machine se kitna door” ko log **level** kehte hain: assembly low-level, Python high-level. (Yeh “level” hamare 4-level stack se alag shabd hai, dhyaan rahe.)

ASLI DUNIYA SE EK EXAMPLE

Aapke phone mein yeh poori seedhi ek saath zinda hai. WhatsApp ke engineers ne high-level bhasha mein likha. Translator ne use machine code banaya. Ab aapka CPU numbers chala raha hai. Koi bhi manzil gayab nahi hui, bas har manzil apne neeche wali ko chhupa deti hai. Abstraction, Chapter 0.4 wala, yahan poori shakal mein khada hai.

YAHAN LOG KYA GALAT SAMAJHTE HAIN

”Computer ab English samajhne laga hai.” Nahi. CPU aaj bhi WAHI numbers samajhta hai jo 1950 mein samajhta tha. Jo badla woh TARJUMA hai: beech ki manzilein itni achhi ho gayin ki insaan ko machine dikhna band ho gayi. Jab AI aapki Hindi se code banata hai, toh bhi aakhir mein wahi 3 aur 7 wale numbers chal rahe hain. Seedhi lambi hui hai, zameen wahi hai.

MAP PE

Kaun kamata hai: har manzil pe alag log. Machine code ke paas duniya ke sabse kam log (chip companies mein, sabse unchi tankhwah prati aadmi). High-level pe crores log (aam software jobs). Aur ab sabse upar wali manzil (AI se code banwana) sabse nayi hai, wahan abhi bheed kam hai aur niyam ban rahe hain. Naya danda khulte hi sabse pehle chadhne wale sabse zyada kamaate hain, har baar yahi hua hai.

KHUD DEKHO (5 minute)

Laptop pe koi bhi website kholo, khali jagah pe right-click karo, “View Page Source” dabao. Jo dikha, woh us page ki recipe hai (HTML). Ghabrao mat, padhna nahi hai. Bas yeh dekho ki har sundar page ke neeche yeh text baitha hai, aur yeh text bhi aakhir mein numbers ban ke hi chala. Parde ke peeche jhaankna aadat bana lo.

SOCHNE KE LIYE

1. (derivation) Machine ko seedha Hindi ya English kyun nahi samjha dete, beech ki bhaashaein hata ke? Kya rukavat hai?

> **Jawab:** Insaan ki bhasha mein matlab aadha bola jaata hai, aadha samjha jaata hai. “Thoda sa daal dena” mein kitna? Machine ko har kadam EXACT chahiye, kyunki uske paas “samajh jaao na” wala mode nahi hai (Chapter 1.5: sab pehle se tay). Isliye beech ki bhasha chahiye jisme matlab ka sirf ek roop ho. AI yahan pehli asli koshish hai: woh aapki adhoori baat ka matlab GUESS karta hai, phir exact code banata hai. Lekin guess kabhi kabhi galat hota hai, isliye AI ke zamaane mein bhi aakhri zimmedari wahi hai: exact kya chahiye, yeh AAPKO pata hona chahiye. Isi liye aap yeh kitaab padh rahe ho.

Chapter 3.2 [SPINE]: Near-English mein likhna

Ab woh cheez saamne dekh lo jisse duniya itna darti hai: code. Yeh raha ek poora, asli program (Python bhasha):

```
price = 300

if price > 500:
    print("mehnga hai")
else:
    print("theek hai")
```

Padho. Aap ise LAGBHAG waise hi padh sakte ho jaise English: “price naam ke dibbe mein 300 rakho. Agar price 500 se bada hai, toh ‘mehnga hai’ dikhao, warna ‘theek hai’ dikhao.” Chalaoge toh screen pe aayega: theek hai.

Bas. Yeh coding hai. Na maths ka pahad, na jaadu. Strict grammar mein likhi recipe.

Ab ek baat jo coding ke bahar ke logon ko koi nahi batata: **har program, WhatsApp se le kar ChatGPT tak, sirf TEEN cheezon se bana hai:**

1. Rakho: naam wale dibbe mein value rakhna (`price = 300`). Dibba wahi RAM ka mez hai (Chapter 2.5), naam insaan ki suvidha ke liye hai.

2. Poochho: shart pe raasta chunna (`if ... else ...`). Yeh wahi “agar” hai jo gates se bana tha (Chapter 1.5), bas ab insaan ki shakal mein.

3. Dohraao: ek kaam baar baar karna (loop):

```
for customer in customer_list:
    bill = bill_nikaalo(customer)
    print(bill)
```

”Har customer ke liye: bill nikaalo, dikhao.” Chahe list mein teen hon ya teen crore, recipe wahi teen lines. YAHI woh jagah hai jahan leverage code mein ghusta hai: insaan teen crore baar nahi kar sakta, loop kar leta hai.

Rakho, poochho, dohraao. Agli baar koi bhi software dekho, khud se poochho: ismein kya rakha ja raha hai, kya poochha ja raha hai, kya dohraaya ja raha hai. Instagram feed: posts ki list pe loop, har post pe “is user ko dikhana banta hai?” wala if, aur dekhe hue posts ka hisaab dibbon mein. Poora app teen cheezon ka jaal.

NAAM

Likhi hui recipe: **code**. Naam wala dibba: **variable**. Shart: **condition** (if/else). Dohraana: **loop**. Strict grammar ka naam: **syntax**. Grammar ki galti: **syntax error**, translator usi waqt pakad leta hai.

ASLI DUNIYA SE EK EXAMPLE

Aapka bank har mahine ki 1 taareekh ko lakhon khaaton pe interest daalta hai. Andar kya hai? Ek loop (“har khaate ke liye”), ek condition (“agar savings account hai”), aur kuch variables (rate, balance, dinon ki ginti). Jo clerk 1960 mein mahina lagata tha (Chapter 1.3), woh aaj 20 lines ka loop hai jo raat ko chalta hai. Duniya ke sabse bade business-software ka pet aise hi seedhe loops se bhara hai.

YAHAN LOG KYA GALAT SAMAJHTE HAIN

”Coding ke liye maths genius chahiye.” Aapne abhi ek program padha aur ek ka andaza lagaya, bina kisi maths ke. Aam software mein jod-ghata se zyada maths hota hi nahi. Asli skill maths nahi, SAAF SOCHNA hai: kaam ko itne chhote, exact kadmon mein todna ki ek bewakoof lekin bijli-tez naukar (machine) unhe kar sake. Jo aadmi business process saaf soch sakta hai, woh coding ki soch pehle se jaanta hai, use sirf grammar nahi aati.

MAP PE

Rupaye ka rasta: duniya mein ~3 crore log code likh ke kamaate hain. Lekin ab asli baat: AI ke baad grammar SASTI ho rahi hai (Claude grammar jaanta hai), aur saaf sochna MEHNGA ho raha hai. Jo aadmi jaanta hai KYA banwana hai aur use exact kadmon mein tod sakta hai, woh ab bina grammar seekhe bhi banwa sakta hai. Aapki is kitaab ki poori bet isi line pe hai.

KHUD DEKHO (5 minute)

Upar wale pehle program mein 300 ko 800 kar do (dimaag mein). Ab kya chhapega? Ab 500 ko 800 karo, price 800 hi rahe. Ab kya? (Dhyaan: “bada hai” ka sawal hai, barabar ka nahi.) Agar dono ka jawab aapne bina atke de diya (pehla: mehnga hai; doosra: theek hai, kyunki 800 > 800 jhooth hai), toh aapne aaj code trace kiya, yehi debugging ki pehli skill hai.

SOCHNE KE LIYE

1. (derivation) Ek bracket ya comma chhootne se poora program kyun ruk jaata hai? Insaan toh aadhi galat spelling waali chitthi bhi padh leta hai.

> **Jawab:** Insaan matlab ka andaza laga leta hai kyunki uske paas duniya ki samajh hai. Translator ke paas sirf grammar ke niyam hain; bracket ke bina use pata hi nahi chalta ki “poochho” kahan khatam hua aur “karo” kahan shuru. Aur GUESS karna jaan-boojh ke nahi rakha gaya: bank ke code mein translator ka galat guess crores ka ho sakta hai. Behtar hai wahin rok do, insaan se poochho. Strict grammar sazaa nahi hai, woh EXACTNESS ki keemat hai, aur exactness hi woh cheez hai jiske liye machine pe bharosa kiya ja sakta hai.

Chapter 3.3 [DEPTH]: Translator ke do tareeke

(DEPTH chapter. Skip kar sakte ho; lautoge toh “compiled vs interpreted” wali har tech baat khul jaayegi.)

Ek kitaab Hindi mein hai, aapko English walon tak pahunchani hai. Do raaste hain, dono aap khud soch sakte ho:

Pehla: poori kitaab ka tarjuma pehle karwa lo. Waqt lagega, lekin uske baad har English reader seedha padhega, poori raftaar se. Aur translator poori kitaab dekhta hai, toh galtiyan pehle hi pakdi jaayengi, chhapne se pehle.

Doosra: reader ke saath ek dubhashiya baitha do. Shuru turant ho jaayega, ek line suno, ek line bolo. Lekin har baar padhne pe tarjuma phir se hoga, toh raftaar kam. Aur galti tabhi pakdegi jab woh line aayegi, beech kitaab mein bhi atak sakte ho.

Code ke bhi yahi do raaste hain:

Compiler = poora tarjuma pehle. Code ek baar machine code mein badla, ab woh file seedha CPU pe bhaagti hai. Tez. Games, browsers, operating systems aise bante hain (bhaashaein: C, C++, Rust).

Interpreter = saath baitha dubhashiya. Likho aur turant chalao, badlo aur phir chalao. Aaraam hai, lekin chalte waqt har line ka tarjuma hota hai, toh dheema (10-100 guna tak). Python aisa hai.

Toh sauda saaf hai: **compiler = mehnat pehle, raftaar baad mein. Interpreter = aaraam pehle, raftaar ki keemat baad mein.** Kaunsa sahi? Galat sawal. KIS KAAM ke liye, yeh poochho. Game ko raftaar chahiye: compiler. Roz badalne wala chhota kaam: interpreter. (Duniya mein beech ke raaste bhi hain, Java aur JavaScript aadha pehle aadha chalte-waqt tarjuma karte hain, lekin idea yehi do hain.)

NAAM

Compiler (poora pehle), **interpreter** (line-by-line). Compiled code ki tayyar file ko log **binary** ya **executable** kehte hain: .exe wali file wahi hai, tarjuma ho chuki recipe.

ASLI DUNIYA SE EK EXAMPLE

Android phone pe kabhi update ke baad likha aata hai “Optimising apps 1 of 132…” Woh yahi hai: phone apps ka tarjuma apne CPU ke liye pehle se kar raha hai, taaki baad mein har app tez khule. Aapne compiler ko kaam karte hue dekha hai, bas naam nahi pata tha.

YAHAN LOG KYA GALAT SAMAJHTE HAIN

”Python dheemi hai toh AI jaise bhaari kaam Python mein kyun hote hain?” Chaal yeh hai: Python sirf STEERING hai. AI ka asli bhaari hisaab (numbers ka guna, arabon baar) compiled code mein likha hai (C/C++ ki libraries), Python bas use bulaata hai. Dheemi bhasha se tez engine chalaya ja raha hai. Abstraction phir se: aaraam upar, raftaar neeche, dono ek saath.

MAP PE

Rupaye ka rasta: server pe har second ka CPU kiraya lagta hai (Part 5 mein poora hisaab). Dheema code = zyada seconds = bada bill. Isliye companies jab badi hoti hain, toh apne sabse zyada chalne wale hisse compiled bhaashaon mein DOBARA likhti hain, sirf bill ghatane ko. Founder ke liye sabak: shuru mein aaraam wali bhasha (raftaar se pehle zinda rehna zaroori hai), scale pe raftaar wali. Galat waqt pe galat chunaav, dono taraf mehnga hai.

KHUD DEKHO (5 minute)

Apne phone/laptop mein files mein jhaanko: .exe ya .apk file (compiled, tarjuma ho chuki, kholo toh ajeeb aksharon ka kachra dikhega kyunki woh machine code hai) vs koi .txt ya website ka source (insaan ke padhne laayak). Do duniyaon ka farq apni aankh se dekh lo: ek insaan ke liye likhi gayi, ek CPU ke liye.

SOCHNE KE LIYE

1. (derivation) Compiler galtiyan pehle pakadta hai, interpreter chalte waqt. Bank ka core system kis se banna chahiye, aur ek naya idea jo har hafte badal raha hai woh kis se? Wajah bhi do.

> **Jawab:** Bank: compiler wali bhasha. Wahan galti chalne ke BAAD milna crores ka ho sakta hai; pehle pakadna hi bachav hai, aur raftaar bhi chahiye. Naya idea: interpreter wali. Wahan sabse bada khatra dheema code nahi, DHEEMA SEEKHNA hai: har hafte badlav karna hai, turant chala ke dekhna hai. Sauda hamesha wahi hai: galti ki keemat vs badlav ki raftaar. Yeh ek line kisi bhi tech decision pe lagao, aadha jawab mil jaayega.

Chapter 3.4 [SPINE]: Itni saari languages kyun hain

Python, Java, C, JavaScript, Swift, Kotlin, Go, Rust… sau se zyada hain. Naye aadmi ko lagta hai yeh pagalpan hai, ya fashion. Nahi. Wajah wahi hai jo gaadiyon ki hai.

Truck, bike, car, tractor: sab “jaane ka saadhan” hain, phir bhi chaar kyun? Kyunki har design ek SAUDA hai: truck saamaan zyada, raftaar kam; bike sasti, baarish mein bheegoge. Ek saadhan sab kuch best nahi kar sakta, kyunki khoobiyaan ek doosre se ladti hain.

Languages ke saude bhi ginti ke hain, chaar mukhya:

Raftaar vs aaraam. Machine ke paas likho (C) toh tez lekin har cheez khud sambhaalo; door likho (Python) toh aaraam lekin dheema. (Pichhle chapter ka sauda.)

Aazaadi vs suraksha. Kuch bhaashaein aapko kuch bhi karne deti hain (C: memory tak seedha haath); taakat milti hai, lekin galti ka darwaza bhi khula. Kuch bandhan lagaati hain (Rust, Java) toh galtiyon ki poori nasl hi khatam.

Aam kaam vs khaas kaam. Kuch har kaam ke liye (Python, Java). Kuch ek hi kaam ke liye bani, lekin us kaam mein raani: SQL sirf data se sawal poochhne ke liye (Part 4 mein milegi), HTML sirf page ki shakal batane ke liye.

Jagah ka kabza. Kuch bhaashaein isliye chalti hain kyunki unka kisi jagah pe kabza hai: browser ke andar sirf JavaScript chalti hai, iPhone app ke liye Apple ne Swift rakhi hai, Android ke liye Kotlin. Wahan kaam karna hai toh wahi bhasha.

Bas. Har language in chaar saudon ka ek jod hai. “Sabse achhi language kaunsi” waisa hi sawal hai jaisa “sabse achhi gaadi kaunsi”: bina kaam bataye jawab hai hi nahi.

NAAM

Ek project mein istemal hui saari bhaashaon aur tools ke jod ko **tech stack** kehte hain. (Chapter 0.3 wala shabd wapas aaya, ab chhote size mein: kaun kis pe khada hai.)

ASLI DUNIYA SE EK EXAMPLE

Zomato ke andar ek saath kam se kam paanch bhaashaein zinda hain: app Kotlin/Swift mein (phones ka kabza), peeche ke servers Java/Go mein (raftaar + bharosa), data ke sawal SQL mein, AI ka kaam Python mein, website JavaScript mein. Koi pagalpan nahi: har jagah wahi bhasha jo us jagah ka sauda jeetti hai. Bade business mein sawaal “kaunsi language” nahi hota, “kis kaam pe kaunsi” hota hai.

YAHAN LOG KYA GALAT SAMAJHTE HAIN

Log language ko dharm bana lete hain (“Python wala hoon main”). Engineers ke jhagde iss pe asli hain, lekin aap founder ki nazar se dekho: language ek HIRING decision hai. Python/JavaScript walon ki bheed badi hai (sasta, jaldi milta hai), Rust walon ki chhoti (mehnga, dhoondhna padega). Naye startup ke liye aadha faisla yahin ho jaata hai, technology se pehle.

MAP PE

Scarcity ka ulta khel dekho: COBOL 1959 ki bhasha hai, koi nahi seekhta. Lekin duniya ke banks ke purane core systems usi pe khade hain, aur woh systems badalna itna khatarnak hai ki koi nahi badalta. Natija: bache-khuche COBOL wale buddhe engineers ko banks moti fees dete hain. Sabak yaad rakho: paisa naye mein bhi hai, aur us puraane mein bhi jise sab chhod gaye lekin duniya jis pe ab bhi khadi hai. Scarcity dono sire pe banti hai.

KHUD DEKHO (5 minute)

Kisi job site pe (LinkedIn/Naukri) search karo: “Python developer” aur phir “COBOL developer”. Ginti dekho, aur jahan salary dikhi ho woh bhi. Ab jo dikha use Chapter 0.2 ke formula mein rakho: demand kitni, supply kitni. Language market ko aapne abhi khud naapa.

SOCHNE KE LIYE

1. (derivation) Browser mein sirf JavaScript chalti hai, yeh “jagah ka kabza” hai. Is kabze se JavaScript walon ki kamaai aur bheed pe kya asar pada hoga? Formula se socho.

> **Jawab:** Kabze ne demand pakki kar di: har website banane wale ko JavaScript chahiye hi chahiye, toh kaam ka size bahut bada. Lekin isi wajah se CRORES logon ne use seekh liya, toh scarcity gir gayi. Natija: kaam sabse zyada, average tankhwah beech ki. Iske ulat, jis skill ki demand badi ho AUR seekhne waale kam hon (aaj: AI systems ko theek se chalana aur jodna), wahan dono dial upar hain. Bheed jahan pahunch chuki hai wahan paisa average ho chuka hota hai; formula hamesha agli khaali jagah dhoondhne ko kehta hai.

Chapter 3.5 [SPINE]: Woh manager jo machine baantta hai

Abhi aapke phone pe kya kya chal raha hai? WhatsApp, gaana, background mein email check, screen ka touch, network ki sunwai. Lekin Part 1 se aap jaante ho: CPU ek waqt mein EK recipe ka EK kadam chalata hai. Toh yeh sab EK SAATH kaise?

Aur ek gehri dikkat bhi hai. Chapter 2.4 ke sawal mein dikha tha: agar ek recipe doosri recipe ki memory mein likh de toh? Sau recipes, ek mez (RAM), koi rok-tok nahi: ek kharab app poore phone ko gira degi.

Toh khud design karo. Aapke paas ek tez CPU hai aur sau recipes hain jo chalna chahti hain. Kya karoge?

Baari baantoh. Har recipe ko thoda sa waqt do: 10 millisecond WhatsApp, 10 gaane ko, 10 screen ko, phir wapas. CPU itna tez hai (arab kadam/second) ki har recipe ko lagta hai machine sirf uski hai. “Ek saath” ek bhram hai, asal mein bijli-tez baari-baari hai.

Deewarein banao. Har recipe ko mez ka apna hissa do, aur niyam: apne hisse ke bahar haath dala toh wahin maar do (yehi “app crash” hai). Ek app mare, baaki bache.

Saajha cheezon ka darbaan bano. Screen ek hai, speaker ek, network ek, storage ek. Recipes ko seedha haath nahi lagane denge; woh darkhaast karengi, darbaan karega. Files ka intezaam bhi yahi darbaan rakhta hai.

Ab dekho aapne kya banaya: ek MALIK recipe, jo baaki sab recipes ke upar baithti hai, waqt baantti hai, deewarein rakhti hai, darbaani karti hai. Yeh operating system hai. Windows, Android, iOS, Linux: yehi chaar naam duniya ki lagbhag har machine ke malik hain.

Aur ab woh baat jo iski asli taakat hai: app likhne wala screen, touch, network, file, KUCH bhi seedha nahi chhoota. Woh sirf OS se maangta hai. Matlab app OS KE LIYE likhi jaati hai, machine ke liye nahi. Isi liye iPhone wali app Android pe nahi chalti: machine lagbhag same hai, MALIK alag hai, aur app malik ki bhasha mein likhi thi.

NAAM

Malik recipe: **operating system (OS)**. Chalti hui recipe ki baari ka intezaam: **scheduling**. Har hardware se baat karne wala OS ka tukda: **driver**. Files ka intezaam: **file system**. Aur app jo OS se maangti hai, us maangne ke tay tareeke ka naam agle chapters mein bada hoga: yehi “API” ka pehla roop hai.

ASLI DUNIYA SE EK EXAMPLE

Ek dhaba socho jisme ek hi chulha hai aur das cooks. Bina manager: jhagda, jala khaana, afra-tafri. Manager aake kya karta hai: chulhe ki baari baantta hai, har cook ka apna saamaan-shelf tay karta hai, aur bhandar ki chaabi apne paas rakhta hai. Dhaba wahi, chulha wahi, lekin ab sau order nikalte hain. OS bilkul yahi manager hai, aur “phone hang” us din hota hai jab manager khud thak jaaye.

YAHAN LOG KYA GALAT SAMAJHTE HAIN

”Android/iOS bas phone ka brand-type hai.” Nahi, woh malik hai, aur malik hona duniya ka sabse bada business-pad hai. Jo OS ka malik hai, woh tay karta hai ki us machine pe kaun sa app aayega, kaise aayega, aur (asli baat) PAISE ka kitna hissa raste mein katega. Apple har app ki kamai ka 15-30% leta hai, sirf darwaze ka malik hone ke naate. Is pad ka naam platform hai, aur platform ka malik hamesha apne upar bane business se zyada sukoon se kamata hai.

MAP PE

Chain dekho: aap 100 rupaye ka in-app purchase karte ho (Level 1 se paisa ghusa). App wale ko ~70-85 mile, Apple/Google ne 15-30 raste mein liye (platform tax). App wala apne cloud aur tools walon ko deta hai. OS ka malik har transaction mein baitha hai, bina woh app banaye. Isliye companies OS/platform ki jang ladti hain marte dum tak: Microsoft (Windows), Google (Android muft baanta, taaki malik bane), Apple. Jo darwaza jeet gaya, woh lagaan wasoolta hai.

KHUD DEKHO (5 minute)

Phone ki Settings kholo, Battery dekho: kaunsi app ne kitna khaya. Phir Apps mein jaake kisi app ki “Permissions” dekho: camera, location, contacts. Yeh dono panne OS ke manager-register hain: kisko kitni baari mili, kisko kaunsi chaabi. Malik ka hisaab-kitaab aap khud padh sakte ho.

SOCHNE KE LIYE

1. (derivation) Google Android ko muft kyun baantta hai, jabki usko banane pe arabon lagte hain? (Chapter 0.1 ki soch lagao: paisa kahan ghusta hai, kahan behta hai.)

> **Jawab:** Kyunki Android bech ke kamaana chhota khel tha, MALIK banna bada. Muft OS se duniya ke 70% phone Google ke darwaze ban gaye: unki search, unka Play Store (15-30% tax), unke ads, unka data. Ek baar banao, arabon phones pe chale, har phone kamaai ka rasta: leverage ka poora dial. Sabak founder ke liye: kabhi kabhi product muft dena hi sabse bada business hota hai, AGAR muft cheez aapko darwaze ka malik banati ho. (Muft ka poora khel Chapter 3.7 mein.)

Chapter 3.6 [DEPTH]: “Running” ka matlab kya hai

(DEPTH chapter. Chhota hai. Iske baad “app band karo, process maro, restart karo” wali har baat ka asli matlab dikhne lagega.)

Ek sawal jo seedha lagta hai lekin nahi hai: WhatsApp aapke phone mein “hai” ka matlab kya hai? Do bilkul alag cheezein hain:

Recipe ki file. Storage (pakki almaari) mein padi hai, ~100 MB. Soyi hui. Bijli jaaye, kuch nahi bigadta. Yeh APP hai.

Chalti hui copy. Jab aapne icon dabaya, OS ne recipe ko RAM ke mez pe utara, uska program counter set kiya, use baari dena shuru kiya. Ab woh ZINDA hai: uski apni memory, uski abhi ki haalat (kaunsi chat khuli hai, kahan tak scroll kiya). Yeh PROCESS hai.

Ek recipe, kai copies bhi ho sakti hain: laptop pe do browser windows = ek program, do process, har ek apni haalat ke saath. Do alag tabs mein do alag email khaate khule hain: recipe saajha, zindagi alag.

Ab roz ki bhasha is naye chashme se padho:

”App kholna” = process janamna (almaari se mez pe). **”App not responding”** = process zinda hai lekin kisi kaam mein atka hai, baari mil rahi hai par jawab nahi de raha. **”Force stop”** = OS process ko maar deta hai. File salaamat. **”Restart karo, theek ho jaayega”** = purani uljhi haalat wali process mari, nayi saaf haalat se janmi. Isliye restart itni beemariyan theek karta hai: beemari HAALAT mein thi, file mein nahi. **”Background apps”** = process zinda hai lekin screen pe nahi; OS use kam baari deta hai, ya soola deta hai.

NAAM

Chalti hui copy: **process**. Uski abhi ki poori haalat (memory

- counter + khuli cheezein): **state**. Yeh shabd bahut bada hai; aage database (Part 4) aur AI (Book 2) mein “state” baar baar aayega. Aur ek process ke andar ki chhoti baari-lene wali dhaaraon ka naam **thread** hai (itna hi kaafi hai abhi).

ASLI DUNIYA SE EK EXAMPLE

Cyber cafe ya kisi saajha computer pe do log alag alag Gmail khol lete hain, do browser windows mein. Recipe (browser) ek hi thi. Lekin har process apna state rakhta hai, isliye do zindagiyan ek machine pe bina mile chal jaati hain. Poora cloud business (Part 5) isi baat pe khada hai: EK machine pe HAZAARON alag processes, har ek apni deewar mein, har ek kisi aur ka kaam karti hui.

YAHAN LOG KYA GALAT SAMAJHTE HAIN

”Maine app band ki, matlab phone se hat gayi.” Ulta bhi: “app delete ki, matlab uska sab kuch gaya.” Dono galat, kyunki log FILE aur PROCESS ko ek samajhte hain. Band karna process ka ant hai, file rahi. Delete karna file ka ant hai, lekin app ka data (chats, settings) aksar alag jagah hai, aur cloud pe bhi. Do naam alag rakho, aadhi confusion khatam.

MAP PE

Rupaye ka rasta: cloud mein aap PROCESS ke waqt ka kiraya dete ho. “Serverless” naam ki cheez mein toh hisaab hi yehi hai: aapka code jitne millisecond process bana raha, utna paisa. Process = meter chalu, process khatam = meter band. Jab Part 5 mein cloud ka bill padhoge, toh har line ke peeche yehi sawal hoga: kitni processes, kitni der, kitne mez (RAM) ke saath.

KHUD DEKHO (5 minute)

Phone pe recent apps ka button dabao (ya laptop pe Task Manager kholo: Ctrl+Shift+Esc). Yeh zinda processes ki list hai. Kisi app ko swipe karke maar do, phir dobara kholo: thoda dheere khuli na, aur pichhli jagah pe nahi? Aapne ek process ka poora janam-mrityu chakra khud chalaya: state gayi, file se naya janam hua.

SOCHNE KE LIYE

1. (derivation) Game beech mein band ho gaya (process mara). Wapas khola toh score wahi ka wahi mila. State toh mar gayi thi, score kaise bacha? Part 1 ke kis chapter ka intezaam hai yeh?

> **Jawab:** Game ne marne se pehle score storage mein likh diya tha (save), yaani tez-bhulakkad RAM se dheemi-pakki almaari mein (Chapter 2.5). Process ki state hamesha maranshil hai; jo bachana ho use pakki jagah likhna hi ekmatra rasta hai. Isi ek kaam ka bada roop DATABASE hai, jo Part 4 ka mukhya kirdaar hai: state ko aisi jagah rakhna jahan process ke marne se, machine ke jalne se, kuch na bigde.

Chapter 3.7 [SPINE]: Free cheezein exist kyun karti hain

Ek ajeeb list dekho: Android muft. WhatsApp muft. Google search muft. Python muft. Linux muft (aur duniya ke lagbhag saare servers us pe chalte hain). Yeh sab banane mein arabon lage. Business ki duniya mein koi arabon laga ke cheez muft kyun baantega?

Chapter 1.6 se chaabi utha lo: software ki COPY muft hoti hai. Toh muft baantna POSSIBLE hai. Lekin possible hona wajah nahi hai. Wajahein chaar hain, aur chaaron mein paisa kahin aur se aa raha hota hai. Har muft cheez pe yeh chaar sawal chalao:

1. Kya muft cheez kisi darwaze ki chaabi hai? Android muft, kyunki har Android phone Google ke ads aur Play Store ka darwaza hai (pichhla chapter). Muft cheez grahak ko us jagah laati hai jahan asli dukan hai.

2. Kya muft aadha hai, poora bikta hai? Muft mein app, paise se extra features. Muft mein 15 GB, paise se 100. Naam: freemium. Lakhon muft users mein se 2-5% khareed lein toh business chal jaata hai, kyunki (phir wahi baat) muft walon ko dene ka kharcha lagbhag zero hai.

3. Kya aap product ho? Facebook/Instagram/YouTube muft hain kyunki asli grahak advertiser hai, aur bikne wali cheez aapka DHYAAN hai. Jo cheez muft hai aur bechne ko kuch dikh nahi raha, wahan aksar aap hi maal ho.

4. Kya sab milkar auzaar banaa rahe hain? Yeh sabse anokha hai: open source. Linux, Python, hazaaron tools: inka code khula hai, koi bhi padhe, sudhaare, muft use kare. Kyun? Kyunki auzaar sabko chahiye aur kisi ka business nahi hai. Companies apne engineers ko inhe sudhaarne ke PAISE deti hain, kyunki unka poora business in auzaaron pe khada hai aur akele banana mehnga padta. Sadak sab milke banate hain, dukaanein apni apni.

NAAM

Khula code: **open source**. Aadha-muft model: **freemium**. Dhyaan bechne ka model: **advertising model**. Aur “muft cheez asli maal ki taraf dhakelti hai” wali soch ka business-naam: **loss leader**.

ASLI DUNIYA SE EK EXAMPLE

Jio ne 2016 mein data lagbhag muft kar diya. Ghaata? Nahi, chaal number 1: pehle poora desh darwaze ke andar, phir dheere se daam, aur upar apps/platforms ka poora mahal. Wahi khel Google ne Android se khela, Amazon delivery ke ghaate se khelta hai. Muft ek hathiyar hai, aur use wohi chala sakta hai jiske paas lambi saans (paisa) aur aage ka naksha ho.

YAHAN LOG KYA GALAT SAMAJHTE HAIN

Do ulti galtiyan. Pehli: “muft hai toh ghatiya hoga.” Linux duniya ka sabse jaancha-parkha software hai; aapka bank, aapka phone (Android Linux pe bana hai), sab us pe hai. Doosri: “muft hai toh muft hai.” Nahi, keemat kahin aur hai: aapka dhyaan, aapka data, ya darwaze ka lagaan. Muft dekho toh hamesha poochho: is chain mein paisa kahan ghus raha hai? (Chapter 0.3 ka sawal, har jagah kaam aata hai.)

MAP PE

Aapke liye iska seedha matlab: aapki startup ka poora auzaar- ghar MUFT hai. Linux, Python, VS Code, Git, arabon dollar ka tool-stack, keemat zero. Pachaas saal pehle business shuru karne ke liye factory chahiye thi; aaj Level 3 ka poora karkhana muft milta hai, aur Level 4 (AI) ka kiraya mahine ke kuch hazaar. Itihaas mein itni sasti entry kabhi nahi thi. Scarcity ab auzaar mein nahi hai; woh IDEA aur EXECUTION mein sarak gayi hai. Auzaar sab ke paas hai, chalana sabko nahi aata.

KHUD DEKHO (5 minute)

Apne phone ki koi 5 muft apps lo. Har ek pe chaar-sawal wala test chalao: darwaza? freemium? aap product ho? open source? Har app kisi ek (ya do) mein baithegi. Jo kisi mein na baithe, usse saavdhaan: ya toh aapko chain dikh nahi rahi, ya woh app data bech rahi hai jo dikhta nahi.

SOCHNE KE LIYE

1. (derivation) WhatsApp ne shuru mein 1 dollar saal ka daam rakha tha, phir Facebook ne khareed ke use bhi hata diya. 19 arab dollar de kar muft kyun kiya? Chain likho.

> **Jawab:** Facebook ka maal dhyaan aur data hai, aur uska sabse bada darr tha ki messaging ka darwaza koi aur (Google, ya khud WhatsApp bada hokar) le le. 19 arab darwaze ki keemat thi, 1 dollar/saal toh rukavat thi jo har naye user ko rokti thi, isliye hatayi: user jitne zyada, network jitna bada, utna mushkil kisi aur ka ghusna. (Iska naam network effect hai, Part 5 mein poora milega.) Sabak: jab koi badi company kisi muft cheez pe arabon lutaaye, toh woh cheez kharch nahi, KILA hai.

SECTION 4: SOFTWARE BANTA KAISE HAI

Paanch chapters. Ab tak dekha ki code kya hai. Ab dekhna hai ki asli software BANTA kaise hai: galtiyon, badlaavon, aur hamesha-galat estimates ke beech.

Yeh aapke business decision mein kahan aayegi: Ek din aap kisi se (insaan ya AI se) software banwaoge. Us din ke liye yeh section hai: bug aane pe kitna gussa jaayaz hai, “ho toh gaya tha” ka matlab kya hai, deadline doguni kyun ho jaati hai, aur paisa kis cheez ka dena chahiye: waade ka ya chalte hue tukde ka. Jo founder yeh nahi jaanta, woh ya toh lutta hai ya galat aadmi pe chillata hai.

Chapter 4.1 [SPINE]: Software mein bug hote hi kyun hain

Pul girte nahi (aksar). Ghadiyan chalti rehti hain. Phir software, jo duniya ke sabse padhe-likhe log banate hain, roz kahin na kahin toot ta kyun rehta hai? Kya yeh laaparwahi hai?

Nahi. Yeh GINTI hai. Khud dekho.

Chapter 1.3 yaad karo: machine wahi karti hai jo LIKHA hai, woh nahi jo aapka MATLAB tha. Toh bug ki definition ek line mein: **jo likha aur jo chaha tha, unke beech ka gap.** Ab sawal yeh hai ki itne hoshiyaar log gap chhodte kyun hain?

Kyunki recipe likhne wale ko HAR situation ka pehle se jawab likhna padta hai. Aur situations ginti mein bam ki tarah phat-ti hain. Ek chhota sa form socho: naam, umar, pincode. Naam khaali ho toh? 500 aksharon ka ho? Emoji ho? Umar 0 ho? Minus 5? 200? Pincode mein akshar hon? User ne aadha bhara aur network gaya? Do baar submit dabaya? Teen fields ke bhi dus-dus roop lo toh hazaar jod ban gaye. Asli app mein aise LAKHON jod hote hain, aur likhne wala insaan har jod apne dimaag mein khel nahi sakta. Jo jod uske dimaag mein kabhi aaya hi nahi, wahi kisi din kisi user ke haath lag jaata hai. Us din “bug mila.”

Ab pul se tulna karo, toh farq dikh jaayega: pul ke paas situations kam hain (gaadi bhaari ya halki, hawa tez ya dheemi, sab ek hi physics ke andar). Software ke paas situations ka visphot hai, aur har naya feature purane sab ke saath naye jod banata hai. Isliye 10 guna bada software 10 guna nahi, 100 guna zyada jodon wala hota hai.

Toh sach yeh hai: **bade software mein bugs ka hona pakka hai, sawal sirf yeh hai ki kitni jaldi milte hain aur kitna nuksan karne se pehle pakde jaate hain.** Achhi company woh nahi jiske bug zero hain (aisi company hai hi nahi), achhi woh hai jiska pakadne aur sudhaarne ka intezaam tez hai. Agla chapter wahi intezaam hai.

NAAM

Gap ka naam **bug** (kissa: 1947 mein ek asli patanga, moth, machine mein phansa mila tha; naam chipak gaya). Anokha, kinaare ka situation: **edge case**. Program ka achanak marna: **crash**. Bug dhoondh ke theek karna: **debugging**.

ASLI DUNIYA SE EK EXAMPLE

31 December ki raat ya cricket final ke waqt UPI/payment apps kyun ladkhadaate hain? Kyunki “ek saath itne log” khud ek edge case hai jo aam din kabhi nahi aata. Ya calendar wale bugs: leap year ka 29 February har chaar saal mein aata hai aur har chaar saal mein kahin na kahin koi system girta hai, kyunki kisi likhne wale ke dimaag mein woh jod nahi aaya tha. Duniya ke sabse bade systems ke bug bhi isi ek kahani ke roop hain: jod jo socha nahi gaya tha.

YAHAN LOG KYA GALAT SAMAJHTE HAIN

”Bug aaya matlab engineer ghatiya hai.” Kabhi kabhi sach, lekin aksar nahi. Naya founder pehle bug pe chillata hai; samajhdaar founder poochhta hai: kitni DER mein pata chala, kitne users tak pahuncha, dobara na ho iska kya intezaam bana? Team ka level bug ki ginti se nahi, JAWAB ki raftaar se napta hai. (Aur jo bechne wala kahe “hamare software mein bug nahi hote,” woh jhooth bol raha hai ya usne bada software banaya hi nahi.)

MAP PE

Bug ka apna bazaar hai. Companies apne system todne walon ko INAAM deti hain: bug bounty. Google/Apple/Microsoft ek sangeen bug ke lakhon-crores tak dete hain, kyunki chori se pehle chowkidaar ko milna sasta hai. India ke ladke bug bounty se ghar baithe dollar kamaate hain: scarcity (todne ki nazar sab ke paas nahi) x size (system arabon logon ke) = inaam bada. Level 3 ke andar yeh ek poora pesha hai jisme degree koi nahi poochhta, sirf hunar.

KHUD DEKHO (5 minute)

Kisi bhi app ke search box mein daalo: kuch nahi (khaali search), sirf space, ek emoji, 200 akshar ka kachra. Dekho kya hota hai. Aksar sab theek hoga (kisi ne yeh jod socha tha), kabhi ajeeb jawab milega (nahi socha tha, aapne edge case pakda). Aap abhi wahi kar rahe ho jo tester tankhwah le kar karta hai: un jodon ko dhoondhna jo likhne wale ke dimaag mein nahi aaye.

SOCHNE KE LIYE

1. (derivation) Do features wale app mein features ke aapas ke jod ginno: 1. Ab 10 features pe kitne jod? Aur is se nikaalo: “bas ek chhota sa feature aur daal do na” pe engineer ka chehra kyun utar jaata hai?

> **Jawab:** 2 features = 1 jod. 10 features = har ek ka har doosre se jod = 45. Feature 5 guna badhe, jod 45 guna. Har naya feature purane SAB ke saath naye jod laata hai, aur har jod ek nayi jagah hai jahan bug chhup sakta hai. Isliye “chhota sa feature” chhota kabhi nahi hota: uski asli keemat uske jodon mein hai. Founder ke liye iska naam hai: feature ki keemat = banane ka waqt + hamesha ke liye badha hua jodon ka bojh. Isliye behtareen products features JODNE se nahi, feature NA jodne ki zid se bante hain.

Chapter 4.2 [DEPTH]: Galti pehle dhoondhna

(DEPTH chapter. Iske bina bhi kahani aage badhegi, lekin jo founder “testing” ka matlab jaanta hai, woh kabhi kachcha maal nahi khareedta.)

Pichhle chapter ka sach maan lo: bugs aayenge hi. Toh ab engineer ki kursi pe baitho aur socho: user tak pahunchne se PEHLE unhe kaise pakdein?

Pehla jawab sab ka wahi hota hai: “chala ke dekh lo.” Theek hai, chalaya, form bhara, kaam kiya. Ship?

Ruko. Aapne EK raasta chala. Pichhle chapter ke hazaar jodon mein se ek. Aur agle hafte jab code badlega (code roz badalta hai), kya aap phir se sab haath se chalaoge? Har badlav ke baad, har raasta, hamesha? Insaan se yeh hota nahi, aur jahan insaan se nahi hota, wahan kya aata hai? (Part 1 se jawab: recipe.)

Toh chaal yeh hai: **jaanchne ka kaam bhi code bana do.** Ek aur recipe likho jo pehli recipe ko chalaye aur jawab jaanche:

```
maan_lo: jod(2, 2) == 4
maan_lo: jod(-5, 5) == 0
maan_lo: bill_nikaalo(khaali_list) == 0
```

Aisi sau-do-sau jaanchein likh do (har edge case jo dimaag mein aaye: khaali, zero, minus, bahut bada). Ab har badlav ke baad machine SAARI jaanchein seconds mein chala deti hai. Kal wala code aaj bhi sahi hai ya kisi ne tod diya, iska jawab har din, muft, turant.

Yeh hi testing ka asli matlab hai: ek baar likho, hamesha jaancho. (Leverage, phir wahi. Achhe engineer ka har auzaar aakhir mein leverage hi nikalta hai.)

Lekin ek imaandaar baat: tests likhna waqt khaata hai, aaj ki raftaar girati hai, kal bachaata hai. Toh kitne tests likhein? Yeh engineering ka nahi, BUSINESS ka sawal hai: galti ki keemat kya hai? Bank ka paisa-ginanne wala code: galti crores ki, tests pe

kanjoosi paagalpan. Naye idea ka pehla version jise 50 log dekhenge: kam tests, raftaar zyaada, theek hai. Jaan boojh ke chuna gaya sauda samajhdaari hai; bina soche chhoda gaya testing laaparwahi hai. Farq wahi “jaan boojh ke” hai.

NAAM

Chhoti jaanchein: **unit tests**. Har badlav pe sab tests apne aap chalna: **CI (continuous integration)**. Naya badlav purani cheez tod de: **regression** (yeh shabd engineers ki zubaan pe roz hota hai: “regression aa gaya”).

ASLI DUNIYA SE EK EXAMPLE

Jab UPI ya bank ka naya version aata hai, woh seedha aap tak nahi aata. Pehle machine pe lakhon nakli transactions chalti hain (har edge case: zero rupaye, ulta time, dohri entry), phir thode asli users pe, phir sab pe. Isliye aapke paise wale apps itne kam girte hain: unke peeche jaanchon ki fauj har raat daudti hai. Bharosa jo aap feel karte ho, woh asal mein tests ki ginti hai.

YAHAN LOG KYA GALAT SAMAJHTE HAIN

”Testing = launch se pehle ek baar sab check kar lena.” Nahi. Woh toh sirf ek raat ka pehra hai. Asli testing STHAAYI hai: code ke saath saath jaanchon ki fauj bhi badhti rehti hai, aur har badlav unse guzar ke hi bahar jaata hai. Isliye jab aap kisi se software banwao, toh poochhna: “tests kitne hain, aur kya woh har badlav pe apne aap chalte hain?” Is ek sawal se aap aadhe kachche vendors chhaant doge.

MAP PE

Kaun kamata hai: QA/test engineers ka poora pesha (India mein lakhs log), aur test-automation ke tools ka apna bazaar. Lekin naya mod dekho: AI ab tests LIKHNE mein sabse achha hai, kyunki edge cases sochna pattern ka kaam hai. Toh “haath se test chalaane wala” pesha sikud raha hai, aur “jaanchon ka intezaam design karne wala” mehnga ho raha hai. Wahi seedhi, phir se: kaam machine ko, soch insaan ko, paisa soch wale ko.

KHUD DEKHO (5 minute)

Login form ke liye 5 aisi jaanchein kaagaz pe likho jo use tod sakti hain (khaali password? emoji wala email? 1000 akshar? do baar submit? net beech mein gaya?). Aapne abhi test plan likha, bina ek line code ke. Banwaate waqt yehi list vendor ko doge toh woh samajh jaayega ki saamne kachcha grahak nahi baitha.

SOCHNE KE LIYE

1. (derivation) Maan lo tests likhna bilkul muft ho jaaye (AI likh de). Kya tab bugs zero ho jaayenge?

> **Jawab:** Nahi. Tests sirf woh jaanchte hain jo kisi ke DIMAAG mein aaya. Bug wahan hota hai jahan kisi ka dimaag gaya hi nahi (Chapter 4.1 ka gap). Muft tests us gap ko chhota karenge, khatam nahi: jo jod socha nahi, uski jaanch bhi nahi likhi jaayegi, AI se bhi tabhi likhegi jab use sochne ka rasta mile. Isliye zero-bug ka waada physics nahi, marketing hai. Ho sakta hai wala best: gap chhota karo, pakadne ki raftaar badhao, nuksan ki had bandho.

Chapter 4.3 [SPINE]: Har badlav ka record

Aapne kabhi aisi files dekhi hain? `project_final.docx`, `project_final_v2.docx`, `project_FINAL_really.docx`. Hasi aati hai, lekin yeh ek asli problem ka kachcha ilaaj hai: cheez badalti rehti hai, aur hume purane roop chahiye hote hain.

Ab software ki duniya mein yeh problem raakshas ban jaati hai. Code roz badalta hai. Das (ya das hazaar) log EK HI code pe kaam kar rahe hain. Kal wala version chalta tha, aaj toota hai: KISNE, KYA, KYUN badla? Aur do logon ne ek hi file ek saath badal di toh kiska badlav rahe?

Khud intezaam design karo. Kya kya chahiye?

1. Har badlav ki photo: kya badla, kisne, kab, aur KYUN (ek line ka note).
2. Kisi bhi purani photo pe wapas jaane ka button.
3. Ek saath kaam: main apni alag copy pe naya feature banaun, tum apni pe bug theek karo, aur baad mein dono ka jod ban jaaye.

Yeh intezaam bana hua hai, naam hai Git (2005, wahi aadmi jisne Linux banaya, apne hi kaam ke liye banaya tha). Uski bhasha teen shabdon mein:

Commit = badlav ki photo + note. “Payment ka bug theek kiya” likh ke photo le li. History mein hamesha ke liye.

Branch = apni alag copy jahan aap bina kisi ko chhede prayog karo. Main branch (asli) safe rehti hai.

Merge = branch ka kaam wapas asli mein jodna. Do logon ne ek hi line badli ho toh Git rok ke poochhta hai: kaunsi rakhein?

Aur GitHub? Woh in sab ka adda hai: internet pe rakhi hui saanjhi history, jahan duniya bhar ki teams (aur open source ka poora sansaar) apna code rakhti hain.

Ek baat dhyaan se: yeh sirf “backup” nahi hai. Yeh HIMMAT ka intezaam hai. Jab har badlav wapas ho sakta hai, toh prayog karne ka darr khatam ho jaata hai. Bina Git ke engineer chhoone se darta hai (“kahin tod na dun”); Git ke saath woh todta hai, seekhta

hai, wapas laata hai. Raftaar ka asli raaz safety net hai.

NAAM

Is poore intezaam ka naam **version control** hai, auzaar ka naam **Git**, adde ka naam **GitHub** (ya GitLab). Do badlaavon ka takraav: **merge conflict**.

ASLI DUNIYA SE EK EXAMPLE

Sabse nazdeeki example aap khud ho: YEH KITAAB ek Git repo mein ban rahi hai. Har chapter ka har badlav commit hai, note ke saath. Purana version 1 poora ka poora archive mein zinda hai, ek command pe wapas aa sakta hai. Jo intezaam Microsoft aur Google ke das hazaar engineers ko sambhaalta hai, wahi aapki kitaab ko sambhaal raha hai, muft. (Chapter 3.7: auzaar sab ke liye ek jaisa muft hai. Farq istemaal mein hai.)

YAHAN LOG KYA GALAT SAMAJHTE HAIN

”Yeh engineers ka andaruni auzaar hai, founder ko kya.” Galat. GitHub kisi engineer ya team ka X-ray hai. Wahan dikhta hai: kaam kitna regular hai (commits), notes kitne saaf hain, kitne log sach mein kaam kar rahe hain. Aur jab aap AI se code banwaoge (aaj ke tools seedha Git ke saath chalte hain), toh commit history hi aapka hisaab-kitaab hai: kya banwaya, kab, kis note ke saath. Register padhna aana chahiye, chahe likhe koi aur.

MAP PE

Company case: Microsoft ne 2018 mein GitHub ko 7.5 arab dollar mein khareeda. Kyun itna, ek “code rakhne ki jagah” ke liye? Kyunki wahan duniya ke engineers ka kaam, aadatein aur aane wale projects dikhte hain: yeh developers ka darwaza hai (Chapter 3.5 ki bhasha mein: platform). Aur ab dekho: usi GitHub ke upar Microsoft ne AI coding tools bechne shuru kiye. Darwaza pehle, dukan baad mein. Pattern har baar wahi hai.

KHUD DEKHO (5 minute)

github.com kholo (bina account ke chalega). Search mein “linux” likho, pehla result kholo. Aap duniya ke sabse zaroori software ka asli code dekh rahe ho. “Commits” pe click karo: lakhon badlaavon ki history, har ek pe naam, taareekh, note. Sabse naya commit shayad kal ka hoga. Yeh 30 saal se roz badal raha hai aur kabhi toota nahi: yehi version control ki taakat hai.

SOCHNE KE LIYE

1. (derivation) Akela aadmi kaam kar raha hai, koi team nahi. Use branch/merge ki kya zaroorat? (Socho: himmat wali baat yahan kaise lagti hai.)

> **Jawab:** Zaroorat wahi hai, roop chhota hai. Branch uska prayogshaala hai: “naya design try karta hoon” wali copy, jisme fail hona muft hai, asli cheez surakshit hai. Aur history uska doosra dimaag hai: chhe mahine baad “yeh ajeeb line kyun likhi thi” ka jawab commit note mein milta hai. Akele aadmi ke paas toh poochhne ko koi doosra hai hi nahi, isliye use record ki zaroorat team se ZYADA hai. (Aapka second-brain repo bhi yahi hai: badalti soch ka version control.)

Chapter 4.4 [DEPTH]: “Mere machine pe toh chal raha tha”

(DEPTH chapter. Yeh line software ki duniya ka sabse purana mazaak hai. Iske peeche ka sach samajh loge toh “deploy” aur “DevOps” jaise bhaari shabd halke ho jaayenge.)

Scene: engineer ne feature banaya, apne laptop pe sau baar chalaya, sab perfect. Server pe daala: crash. User ke phone pe: khali screen. Engineer ki pehli line: “mere machine pe toh chal raha tha…”

Woh jhooth nahi bol raha. Toh galti kahan hai?

Chapter 3.5 se socho. Recipe kabhi akeli nahi chalti: woh OS se maangti hai, doosri recipes (libraries) ko bulaati hai, settings padhti hai, files dhoondhti hai. Matlab recipe ke aas paas ek poora MAAHAUL hai jis pe woh tiki hai:

```
code khud            (yeh toh dono jagah same tha)
+ OS aur uska version     (laptop: Windows 11; server: Linux)
+ libraries ke versions   (laptop pe nayi, server pe purani)
+ settings, passwords     (laptop pe test wale, server pe asli)
+ doosri services         (laptop pe nakli bank; server pe asli)
```

Code same, maahaul alag = alag natija. “Chal raha tha” ka poora sach hai: “MERE maahaul mein chal raha tha.”

Toh ilaaj khud soch lo: maahaul ko bhi code jaisa bana do. Likh do ki is recipe ko kya kya chahiye (OS, libraries ke EXACT versions, settings), aur aisa intezaam karo ki har jagah, laptop se server tak, WAHI maahaul ban jaaye. Recipe ke saath poora kitchen hi pack kar do.

Yeh intezaam bana: pehle list-waali files (requirements), phir poora packed-kitchen: container. Ek dibba jisme code + uska maahaul saath band hai; dibba jahan bhi khulega, andar ki duniya same hogi. Ship ke container se hi naam liya gaya: saamaan koi bhi ho, dibba ek jaisa, isliye har port ka crane use utha leta hai.

NAAM

Recipe ke aas paas ki poori duniya: **environment**. Packed kitchen wala dibba: **container**, sabse mashhoor auzaar: **Docker**. Code ko laptop se asli server tak pahunchana: **deploy**. Aur yeh sab intezaam sambhaalne ka pesha: **DevOps**.

ASLI DUNIYA SE EK EXAMPLE

Aapke ghar ki dal har baar perfect banti hai. Wahi recipe mausi ke ghar banao: cooker alag, aanch alag, namak ka dabba alag. Swad badla. Recipe jhooth nahi boli, kitchen badla tha. Ab restaurant chains ka raaz dekho: McDonald’s har jagah ek jaisa isliye hai kyunki unhone KITCHEN ko standard kiya, sirf recipe ko nahi. Docker software ka McDonald’s-kitchen hai.

YAHAN LOG KYA GALAT SAMAJHTE HAIN

”Software = code ki file. File bhej di, kaam khatam.” Ab aap jaante ho: software = code + maahaul. Isliye “app bana di, ab live karna toh bas ek button hoga na” founder ka sabse mehnga bhram hai. Deploy apna poora kaam hai: maahaul banana, jaanchna, purane version se naye pe bina rukawat jaana, galti pe wapas aana. Estimate mein iska waqt alag se poochho, warna “ho gaya” aur “chal raha hai” ke beech ke hafte aapko surprise karenge.

MAP PE

Rupaye ka rasta: yeh dard itna asli hai ki iske ilaaj ki companies arabon ki hain. Docker ne dibba diya; phir sawal aaya “hazaar dibbon ko kaun sambhaale” toh Google ne Kubernetes khola (open source: Chapter 3.7 ki chaal, taaki sab uske cloud ke tareeke pe aa jaayein); aur cloud companies (Part 5) ka aadha business hi yeh hai: “maahaul ka jhanjhat humein do, kiraya do.” Jahan har team ka dard ek jaisa ho, wahan platform banta hai, aur platform lagaan wasoolta hai. Dard dhoondho, dukan wahin kholo.

KHUD DEKHO (5 minute)

Play Store/App Store mein kisi app ka page kholo, neeche “Requires Android 12+” ya “Requires iOS 16+” dhoondho. Yeh environment ki sabse seedhi shakal hai: app keh rahi hai “mera kitchen kam se kam itna naya ho.” Ab samajh aayega purane phone pe nayi app kyun nahi chalti: code nahi, maahaul chhota pad gaya.

SOCHNE KE LIYE

1. (derivation) Do dost, bilkul same phone model, same app version. Ek ke phone pe app roz crash, doosre pe kabhi nahi. Code same, phone same. Ab kya alag ho sakta hai? Kam se kam teen cheezein ginno.

> **Jawab:** Maahaul ab bhi alag hai: (1) OS ka chhota version/update alag ho sakta hai, (2) ek ka storage lagbhag full hai (likhne ki jagah nahi = ajeeb crashes), (3) permissions alag (ek ne location mana kiya, app ka woh raasta kabhi test nahi hua tha), (4) bhasha/region setting alag (date ka format alag = parse error), (5) background apps aur battery-saver alag. Sabak: “same phone” kabhi same nahi hota. Isliye bade apps hazaaron asli devices pe test hote hain, aur phir bhi Chapter 4.1 ka niyam lagta hai: koi na koi jod chhoot hi jaata hai.

Chapter 4.5 [SPINE]: Estimate hamesha galat kyun hota hai

Duniya ka har founder ek hi kahani sunaata hai: “engineer ne kaha do hafte. Do mahine ho gaye.” Ghar banwane wale bhi yahi kahaani sunaate hain, lekin software mein yeh itna PAKKA kyun hai? Kya sab engineers jhoothhe hain?

Nahi. Wajah software ke swabhaav mein hai, aur aap use Part 1 ke ek sach se khud nikaal sakte ho.

Chapter 1.6: **copy muft hai.** Toh jo software pehle se bana hua hai, use dobara koi nahi banata, copy kar lete hain (ya khareed lete hain, ya open source utha lete hain). Iska ulta matlab dhyaan se socho: **software banane ka kaam sirf tab hota hai jab woh cheez pehle kabhi bani hi nahi.** Har software project, definition se, kuch NAYA banana hai.

Ab tulna karo. Thekedaar sauvan ghar bana raha hai: har deewar pehle jaisi, surprises kam, estimate theek baithta hai. Engineer har baar apna PEHLA “yeh wala” bana raha hai. Aur naye kaam mein surprises kahan chhupe hain, yeh pehle se dikhta nahi (dikh jaata toh surprise na hota). Estimate galat isliye hota hai kyunki woh us cheez ka naap hai jo abhi andhere mein hai.

Teen aur cheezein andhera gehra karti hain:

”90% ho gaya” ka bhram. Dikhne wala hissa (screens, buttons) pehle banta hai aur jaldi banta hai. Na-dikhne wala (edge cases, deploy, doosre systems se jod: Chapter 4.1 aur 4.4 ke saare raakshas) baad mein aata hai. Isliye “90% done” aksar aadhe raaste ka naam hai: bacha hua 10% dikhta chhota hai, hota bada hai.

Badalti maang. Aadha banne ke baad founder use pehli baar chalata hai aur samajh aata hai ki asal mein chahiye kya tha. Maang badli, naap purana: estimate ka kya kasoor?

Jod ka dard. Do bane-banaye tukde jodna “bas jod do” nahi hota (Chapter 4.4: har tukde ka apna maahaul). Jod hamesha naap se zyada khaata hai.

Toh ilaaj kya hai? Estimate sudhaarna nahi (woh kabhi poora nahi sudhrega), KAAM KA TAREEKA badalna:

Chhote tukde, chalta hua maal. Chhe mahine ka waada mat lo; do hafte ka aisa tukda maango jo CHAL ke dikhe. Chalta tukda jhooth nahi bolta. Har tukde ke baad andhera thoda kam, agla estimate thoda sahi. Aur sabse pehle woh tukda banao jo idea ki sabse badi shanka ko test kare, sabse sundar screen nahi.

NAAM

Chhote-tukdon wale tareeke ka naam **agile** hai (ceremonies bahut hain, asli baat wahi tukde hain). Pehla chhota chalta roop: **MVP (minimum viable product)**. Aur “kaam apne diye gaye waqt se hamesha zyada leta hai, is niyam ko jaanne ke baad bhi” ke mazaak ka naam **Hofstadter’s Law** hai.

ASLI DUNIYA SE EK EXAMPLE

Duniya ke bade software haadse yehi kahani hain: mahino ka waada, saalon ki deri, kabhi kabhi poora project hi radd (sarkari systems mein aam, arabon dubte hain). Aur ulti kahani bhi utni hi sach hai: WhatsApp, Instagram, Zerodha sab ne pehle ek CHHOTA chalta hua tukda nikala tha (Instagram toh ek badi app ka kata hua ek feature tha: sirf photo, filter, share). Tukda chala, tab badhaya. Bade waade doobte hain, chhote tukde tairte hain.

YAHAN LOG KYA GALAT SAMAJHTE HAIN

”Late matlab team nikammi ya chor.” Ab aap jaante ho: naye kaam ka naap hi andhere ka naap hai. Nikamma hone ki asli nishaaniyan doosri hain: chalta hua tukda kabhi na dikhana, “sab ek saath aakhri mein aayega” kehna, har hafte wahi “almost done.” Aur imaandaar team ki nishaani: chhote tukde, har tukda chalta hua, aur bura news JALDI batana. Ab aap dono ko pehchaan sakte ho.

MAP PE

Yeh chapter seedha aapke batue se juda hai. Jab aap software banwao (insaan se ya AI se): (1) paisa waade pe nahi, chalte tukde pe do; (2) pehla tukda woh maango jo sabse badi shanka test kare; (3) “90% done” suno toh demo maango, screen nahi, ASLI chalta system; (4) estimate ko 2-3 se guna kar ke apna plan banao, aur jo team khud aisa kar ke bataye use izzat do, woh imaandaar hai. Yeh chaar line aapko un founders se alag karti hai jo lakhs luta ke seekhte hain.

KHUD DEKHO (5 minute)

Apna koi bhi purana andaaza yaad karo: shaadi ki taiyari, ghar shift karna, koi bhi naya kaam. Kitna socha tha, kitna laga? Anupaat nikaalo (aksar 1.5x-3x). Ab dhyaan do: jo kaam aap PEHLI BAAR kar rahe the, wahi sabse zyada phisle. Naye kaam ka andhera sirf software ka nahi, insaan ka niyam hai; software mein bas HAR kaam naya hota hai.

SOCHNE KE LIYE

1. (derivation) AI ab code ke bade hisse minutes mein likh deta hai. Toh kya ab estimates sahi ho jaayenge aur software waqt pe banega?

> **Jawab:** Thoda behtar, poora nahi. AI ne LIKHNE ka waqt giraya hai, lekin estimate ka andhera likhne mein tha hi kam: woh SAMAJHNE mein hai (kya chahiye), JODNE mein (maahaul, doosre systems), aur JAANCHNE mein (kaunsa jod chhoota). Balki naya khatra aaya hai: AI se “90% dikhne wala” ab ghanton mein ban jaata hai, toh bhram aur tez banta hai. Founder ka niyam wahi rahega: chalta tukda hi sach hai. Haan, ek badlav asli hai: chhota tukda ab SASTA ban-ta hai, toh prayog zyada kar sakte ho, galat raasta jaldi chhod sakte ho. AI estimate theek nahi karta, galti ki keemat girata hai.

PART 2 KA ANT: NAKSHA AUR AGLA KADAM

Jo aapke paas ab hai

```
TARJUMA KI SEEDHI    machine code -> assembly -> languages -> AI
                     har manzil neeche wali ko chhupaati hai
TEEN CHEEZEIN        rakho (variable), poochho (if), dohraao (loop):
                     har program inhi ka jaal
DO TRANSLATOR        compiler (pehle, tez) vs interpreter (saath
                     saath, aaraam): galti ki keemat vs badlav
                     ki raftaar ka sauda
LANGUAGES            har ek chaar saudon ka jod; "kaunsi best"
                     nahi, "kis kaam pe kaunsi"
OS                   machine ka malik: baari, deewar, darbaani;
                     malik lagaan wasoolta hai (platform tax)
PROCESS              file soyi recipe, process zinda copy apni
                     state ke saath; restart = nayi saaf state
MUFT KA KHEL         darwaza / freemium / aap-product-ho /
                     open source: paisa hamesha kahin aur se
BUG                  likha vs chaha ka gap; jodon ka visphot;
                     ginti hai, laaparwahi nahi
TESTING              jaanchna bhi code hai; kitna, yeh business
                     sawal hai: galti ki keemat kya hai
VERSION CONTROL      har badlav ki photo + wapas jaane ka button
                     = himmat ka intezaam (Git, GitHub)
ENVIRONMENT          software = code + maahaul; isliye "mere
                     machine pe chal raha tha" (Docker, deploy)
ESTIMATE             har software pehli baar ban raha hai;
                     ilaaj naap nahi, chhote chalte tukde (MVP)
```

Ek test, khud ke liye

Bina kitaab palte, bol ke jawab do:

1. iPhone wali app Android pe kyun nahi chalti?
2. Google Android muft kyun baantta hai?
3. ”Bug-free software” ka waada jhooth kyun hai?

4. Software banwaate waqt paisa kis cheez pe dena chahiye: waade pe ya kis pe? Aur kyun?

Atke toh: 1 -> 3.5, 2 -> 3.5/3.7, 3 -> 4.1/4.2, 4 -> 4.5.

Part 3 mein kya hai

Ab tak ki poori kahani EK machine ke andar thi. Lekin aapka har message doosri machine tak jaata hai: shehar paar, samundar paar, aadhe second mein. Kaise? Machines ek doosre ko dhoondhti kaise hain? Raaste mein kaun kaun baitha hai? “Public road pe private baat” (aapka password!) kaise bachti hai? Aur internet ka malik aakhir kaun hai? Part 3: MACHINES KA JAAL.

PART 2 KA MINI-GLOSSARY

agile	chhote chalte tukdon mein banane ka tareeka (4.5)
assembly	numbers ke naam wali bhasha, machine ke paas (3.1)
binary/executable	tarjuma ho chuki, seedha chalne wali file (3.3)
branch / merge	apni prayog-copy / use wapas jodna (4.3)
bug	jo likha aur jo chaha, unke beech ka gap (4.1)
CI	har badlav pe saare tests apne aap chalna (4.2)
code	strict grammar mein likhi recipe (3.2)
commit	badlav ki photo + kyun ka note (4.3)
compiler	poora tarjuma pehle karne wala (3.3)
condition (if)	shart pe raasta chunna (3.2)
container/Docker	code + maahaul ek dibbe mein band (4.4)
crash	program ka achanak marna (4.1)
debugging	bug dhoondh ke theek karna (4.1)
deploy	code ko asli server tak pahunchana (4.4)
DevOps	deploy aur maahaul ka intezaam-pesha (4.4)
driver	hardware se baat karne wala OS ka tukda (3.5)
edge case	kinaare ka anokha situation (4.1)
environment	recipe ke aas paas ki poori duniya (4.4)
file system	OS ka files ka intezaam (3.5)
freemium	muft aadha, poora bikta hai (3.7)
Git / GitHub	version control ka auzaar / uska adda (4.3)
interpreter	line-by-line saath chalne wala dubhashiya (3.3)
loop	ek kaam baar baar; leverage code mein yahin (3.2)
machine code	numbers wali bhasha jo CPU seedha samajhta (3.1)
merge conflict	do badlaavon ka takraav (4.3)
MVP	pehla chhota CHALTA roop (4.5)
open source	khula code, sab ka saanjha auzaar (3.7)
OS	machine ka malik: baari, deewar, darbaan (3.5)
platform (tax)	darwaze ka malik aur uska lagaan (3.5)
process / state	zinda copy / uski abhi ki poori haalat (3.6)
regression	naye badlav ne purana kuch tod diya (4.2)
scheduling	CPU ki baari baantna (3.5)
syntax (error)	strict grammar (ki galti) (3.2)
tech stack	project ke tools/bhaashaon ka poora jod (3.4)
thread	process ke andar ki chhoti dhaara (3.6)
unit test	chhoti apne-aap chalne wali jaanch (4.2)
variable	naam wala dibba (3.2)
version control	badlaavon ki history ka intezaam (4.3)